

[HTTPS://JDBOSTONBU-OPS.GITHUB.IO/MINI-TASK-APP/](https://jdbostonbu-ops.github.io/mini-task-app/)

EXERCISE 1: WRITE YOUR FIRST FUNCTIONS IN THE CONSOLE (LEVEL 1, ~25 MIN)

No HTML page needed. Open DevTools > Console (Cmd+Option+I, then click Console). Type each function below directly in the console and test it.

```
function isEven(n) {  
    return n % 2 === 0;  
}  
  
function isSquare(n) {  
    // Check if the square root of n is an integer  
    return Math.sqrt(n) % 1 === 0;  
}  
  
function shout(message) {  
    return message.toUpperCase();  
}  
  
function isLong(text) {  
    return text.length > 5;  
}  
  
console.log(isSquare(16)); // true  
console.log (isEven(4));  // true  
console.log (isEven(7));  // false  
console.log(shout("jacqueline"));  
console.log(isLong("bananas"));
```

EXERCISE 2: LOOP OVER AN ARRAY (LEVEL 1, ~30 MIN)

Step 1: Create a new file `loops.html` in your `week-02` folder, with a `<script>` at the bottom of the body. Open DevTools > Console.

```
const names = ["Alex", "Sam", "Jordan", "Riley", "Casey"];

for (const name of names) {
  console.log(` Hello, ${name}!` );
}

// "Hello, Alex!"
// "Hello, Sam!"
// "Hello, Jordan!"
// "Hello, Riley!"
// "Hello, Casey!"

const greetings = []; // greetings is array

// Step 3: Classic for loop with an index
for (let i = 0; i < names.length; i++) {
  console.log(` ${i + 1}. Hello, ${names[i]}!` );
}

// Step 4 & 5: for...of loop with character filter and building the new array
let count = 1 // declare the variable

for (const name of names) {
  if (name.length >= 4) {
    const message = ` ${count}. Hello, ${name}!` ; // Add the    // number here
    console.log(message);
    greetings.push(message); // Adds them to our new array
    count++; // adds +1
  }
}
```

```
}
```

```
// Log the final array
```

```
console.log(greetings);// Array ["1. Hello, Alex!", "2. Hello, Jordan!", "3. Hello, Riley!", "4. Hello, Casey!"]
```

EXERCISE 3: BUILD A LIST DYNAMICALLY FROM AN ARRAY OF OBJECTS (LEVEL 2, ~35 MIN)

This connects today's JS fundamentals back to the DOM.

Step 3: Modify:

```
const books = [
  { title: "Atomic Habits", author: "James Clear" },
  { title: "Educated", author: "Tara Westover" },
  { title: "The Mountain Is You", author: "Brianna Wiest" },
  { title: "Becoming", author: "Michelle Obama" },
];
books.push(
  { title: "Brain Games", author: "John Doe" },
  { title: "Deep Work", author: "Cal Newport" }
);

const list = document.getElementById("book-list");

for (const book of books) {
  const li = document.createElement("li");
  li.textContent = `${book.title} ${book.author}`;
  list.appendChild(li);
}
```

```
const books = [
  { title: "Atomic Habits", author: "James Clear" },
  { title: "Educated", author: "Tara Westover" },
  { title: "The Mountain Is You", author: "Brianna Wiest" },
  { title: "Becoming", author: "Michelle Obama" }
];

const newBooks = [
  { title: "Brain Games", author: "John Doe" },
  { title: "Deep Work", author: "Cal Newport" }
];

function renderBooks(bookArray) {
  const list = document.getElementById("book-list");
  // Bonus: Clear the list before rendering
  list.innerHTML = "";
```

```

for (const book of bookArray) {
  const li = document.createElement("li");
  li.innerHTML = `<strong>${book.title}</strong> ${book.author}`;
  list.appendChild(li);
}
}
// First call: Renders the original 4 books
renderBooks(books);

// Second call
renderBooks(newBooks); // 2 Books display on the screen

```

EXERCISE 4: "AI, REFACTOR MY CODE" (LEVEL 2, ~30 MIN)

The accept/reject/modify workflow, applied to functions and structure.

Step 1: Take the messy procedural code below. Save it as `messy.js`:

Step 2: Send to ChatGPT:

AI's Change, What it Does, My Decision (Accept/Reject/Modify, Why?

Moved scores in to an array, single source of data, accept, cleaner code.

Created `getAverage(numArray)` function, Reusable, accept, good practice.

Used a for loop(`const score of numArray`) in both functions, reusable, accept, Good practice

```

sum += score to add up all the scores, accept, good practice
Return sum/numArray.length, to determin the average, usable, good practice

```

Finds the highest number by using a function (`numArray`) =>, readable, accept, good practice

Used for ...of, easy to read, acceptable, good practice

Used if statement, `score > highest` return highest, accept, good practice

Used `console.log` to display average and highest score, correct; math was tested in the console, accept, good practice.

Step 4: Apply your decisions. Run the result. Does it still output the same numbers? Yes, it was correct.

EXERCISE 5: MINI-APP -- BUILD A TODO LIST (LEVEL 3, ~40 MIN)

Combine functions + arrays + conditionals + DOM updates into one small app.

I had to modify the code from `innerHTML` to `textContent` and `span` which was unnecessary in the code.